

# Integrating Large Language Models into Deterministic Compiler Optimization Pipelines

Girish Rawat

*MSc Artificial Intelligence*

*Queen Mary University of London*

London, UK

ec25122@qmul.ac.uk

**Abstract**—The application of Large Language Models to software engineering has yielded significant advancements in code generation and review. However, using these models for compiler optimizations remains challenging due to their propensity for hallucinations and generating semantically incorrect code. This dissertation presents a novel architecture that integrates probabilistic Large Language Models into a deterministic compiler pipeline. By utilizing the LLVM Intermediate Representation and formal verification tools like Alive2, the proposed orchestrator mathematically proves the semantic equivalence of generated optimizations before applying them. If an optimization fails verification, the pipeline safely falls back to the original unoptimized code. Evaluated on both synthetic data and real world utilities, the system demonstrates robust fail closed behavior. While small local models struggled to adhere to strict code output formats, the architectural framework proved entirely successful at isolating failures, ensuring that the final compiled binary is valid by construction. This work establishes a foundation for safely applying frontier AI models to low level compiler optimizations.

**Index Terms**—Compiler Optimization, Large Language Models, LLVM, Formal Verification, Alive2, Translation Validation

## I. INTRODUCTION

Compilers are the bedrock of modern software engineering. They translate human readable source code into highly optimized machine code. Traditional compiler optimization passes rely on hardcoded heuristics and graph transformations designed by human experts. While effective, these hand crafted rules often miss highly context specific optimization opportunities.

Large Language Models have recently demonstrated an unprecedented ability to understand and generate code. This raises a compelling research question: can we leverage the pattern recognition capabilities of these models to discover novel compiler optimizations that traditional heuristics miss?

The primary barrier to answering this question is reliability. Compilers demand absolute semantic correctness. A single incorrect instruction transformation can introduce catastrophic bugs into software. Large Language Models are probabilistic by nature and are known to hallucinate. They cannot be trusted blindly with low level compilation.

This dissertation introduces a solution to this reliability problem. It proposes a deterministic state machine orchestrator that safely harnesses probabilistic models. The core idea is

to treat the AI model as an untrusted oracle. The model suggests an optimization, and a formal verification theorem prover checks the suggestion. Only mathematically proven optimizations are permitted into the final binary. This paper details the design, implementation, and evaluation of this pipeline, focusing on its ability to handle real world code without compromising structural integrity.

## II. BACKGROUND

### A. LLVM Intermediate Representation

LLVM is a widely used compiler infrastructure project [1]. It operates around a central Intermediate Representation (IR). Code written in C, C++, or Rust is first compiled by a front end into LLVM IR. The LLVM optimizer then runs various passes over this IR to improve its efficiency. Finally, a back end translates the optimized IR into machine code for a specific architecture.

LLVM IR is strongly typed and uses Static Single Assignment form, meaning each variable is assigned exactly once. This structure makes it ideal for programmatic analysis and transformation. The pipeline developed in this project intercepts the compilation process at the IR level, specifically targeting unoptimized code generated at the “-O0” optimization level.

### B. Formal Verification and Alive2

Formal verification uses mathematical proofs to guarantee that a system satisfies certain properties. In the context of compilers, translation validation ensures that an optimized piece of code behaves exactly the same as the original code under all possible inputs.

Alive2 is a state of the art translation validation tool for LLVM IR [2]. It uses Satisfiability Modulo Theories (SMT) solvers to formally prove refinement. If an optimized function is a valid refinement of the original function, Alive2 mathematically guarantees that no new undefined behavior has been introduced. If the optimization is invalid, Alive2 provides a concrete counterexample. This pipeline relies on Alive2 to act as an infallible gatekeeper for AI generated code.

### C. Large Language Models in Code Generation

Recent models like GPT-4, Claude, and specialized coding models like Qwen Coder [3] have shown remarkable proficiency in high level languages like Python and JavaScript.

However, LLVM IR is a verbose and highly structured assembly like language. Prompting an AI to modify LLVM IR zero shot requires the model to understand complex use def chains and strict syntax rules.

### III. METHODOLOGY: PIPELINE ARCHITECTURE

The core contribution of this project is a robust six phase orchestrator. It manages the lifecycle of each function as it is parsed, optimized, verified, and reassembled.

#### A. Phase 1: Parsing and Standalone Extraction

The pipeline begins by ingesting an entire LLVM module containing unoptimized IR. Because functions often reference global variables or other functions, they cannot be compiled in isolation. Phase 1 safely extracts each function definition and prepends a shared module preamble. This preamble contains the target datalayout, type definitions, and declarations of sibling functions. The result is a set of standalone IR blocks that can be individually assembled and verified.

#### B. Phase 2: Triage

Not all functions benefit from AI optimization. Very simple functions (such as basic getters or setters) do not offer enough complexity to warrant the computational cost of an AI inference call. Phase 2 calculates the cyclomatic complexity of each extracted function. Functions falling below a configurable threshold are marked as triaged and skip the optimization phases entirely. This phase also counts the tokens in the function to determine which model tier is most appropriate.

#### C. Phase 3: Routing and Inference

Functions that pass the triage phase are routed to an available AI model. The system supports local inference using Ollama, which allows for zero cost privacy preserving experimentation. The pipeline constructs a highly specific prompt instructing the model to reduce instruction count and simplify control flow while preserving semantic behavior. It utilizes a one shot prompting strategy, providing the model with a clear example of an unoptimized function and its correct raw output format. The model is instructed to return only the optimized code block.

#### D. Phase 4: Candidate Reconstruction

Once the model returns a response, Phase 4 attempts to extract the raw LLVM IR from the text. If the extraction is successful, the candidate function body is injected back into the standalone module context created in Phase 1. This reconstructed candidate is now ready for syntax checking and formal verification. If the model fails to return valid syntax, this phase flags the candidate as invalid.

#### E. Phase 5: Formal Verification Gate

This is the most critical phase of the orchestrator. The reconstructed candidate is first passed through `llvm-as` to verify structural and syntactic correctness. If it passes, the candidate and the original function are fed into the Alive2

toolchain. Alive2 attempts to prove that the candidate is a valid refinement of the original.

Based on the SMT solver results, the function receives a strict verdict. A “PASSED” verdict means the optimization is formally proven. A “REJECTED” verdict means Alive2 found a counterexample where the behavior diverges. “SYNTAX\_FAIL” indicates the model wrote invalid LLVM IR.

#### F. Phase 6: Assembly and Compilation

The final phase reconstructs the entire LLVM module. It iterates through every function and checks its verification verdict. If a function received a PASSED verdict, the orchestrator locks in the AI optimized body. For any other verdict (including timeouts, syntax failures, or rejections), the orchestrator discards the AI output and seamlessly falls back to the original unoptimized code.

Finally, the reassembled module is written to a temporary file and compiled into a native binary executable using the system compiler toolchain. Because the fallback mechanism is absolute, the resulting binary is guaranteed to be valid by construction.

## IV. EXPERIMENTAL SETUP

To evaluate the pipeline, two distinct datasets were used.

The first dataset consisted of synthetic functions designed specifically to contain obvious optimization opportunities, such as redundant arithmetic operations (for example, adding zero to a variable) and dead code blocks. This dataset served as a baseline to ensure the pipeline logic and verification tools functioned correctly.

The second dataset consisted of real world C code compiled to LLVM IR. The `sbase` corpus, a collection of standard Unix utilities, was chosen for this task. It contains hundreds of realistic functions of varying complexity, providing a robust stress test for both the orchestrator and the local AI models.

The experiments were run locally using the Ollama inference server. The `qwen2.5-coder` family of models (specifically the 3B and 7B parameter variants) were selected for their coding proficiency and ability to run on consumer hardware.

## V. RESULTS AND DISCUSSION

#### A. Synthetic Data Validation

The pipeline performed exceptionally well on the synthetic dataset. The local models were able to identify the redundant arithmetic operations and output simplified LLVM IR. Phase 5 successfully invoked Alive2, which formally verified the mathematical equivalency of the optimizations. For functions that passed verification, the orchestrator recorded an average instruction reduction of 78 percent. This confirmed that the state machine, the verification gate, and the fallback logic were completely sound.

### B. Real World Evaluation on the sbase Corpus

The evaluation on the real world `sbase` corpus revealed significant challenges regarding model capability. A total of 311 functions were processed. The pipeline successfully parsed the module, triaged the simple functions, and routed the complex functions to the local models.

However, the local 3B and 7B models struggled severely with the instruction following constraints when presented with large, complex real world IR. Despite strict system prompts and one shot examples explicitly requesting raw code output, the models consistently generated conversational English explanations of the code rather than optimized LLVM IR.

For example, when tasked with optimizing the main function of the `tar` utility, the model returned prose explaining that the code handled command line arguments and directory traversal, completely omitting any valid `define` blocks.

### C. Pipeline Robustness

While the local models failed to generate valid optimizations for the `sbase` corpus, the pipeline orchestrator performed flawlessly. Every single malformed response from the AI was caught during the sanitization or syntax checking phases. The functions were correctly flagged with “SYNTAX\_FAIL” or “PENDING” verdicts.

Phase 6 executed its fallback logic perfectly, reverting all 266 evaluated functions to their original, unoptimized states. The final assembled module was successfully compiled into a working binary. This is a highly successful result from an architectural standpoint. It proves that the fail closed design works exactly as intended. The pipeline can safely absorb complete hallucinations and formatting failures from an untrusted model without breaking the build process or introducing bugs into the final software.

## VI. FUTURE WORK

The current limitation lies entirely in the instruction following capability of small local models. The immediate next step is to evaluate this pipeline using frontier API models such as GPT-4o or Claude 3.5 Sonnet. These models possess vastly superior context windows and adherence to structural prompts. Plugging a frontier model into the existing, proven orchestrator will allow us to measure the true optimization ceiling of this approach.

Furthermore, future evaluations should integrate the pipeline with industry standard benchmarking frameworks like the SPEC CPU suite. By compiling the SPEC benchmarks with a standard compiler, capturing the IR, passing it through our AI orchestrator, and benchmarking the resulting binary, we can directly quantify the performance gains (or regressions) against traditional compilation techniques.

## VII. CONCLUSION

This dissertation presented a deterministic, fail safe pipeline for integrating Large Language Models into the compiler optimization process. By combining the probabilistic nature of AI with the absolute certainty of formal SMT verification,

the orchestrator ensures that no semantic errors can ever reach the final compiled binary.

Experimental results demonstrated that while small local models currently lack the capability to reliably manipulate complex LLVM IR zero shot, the architectural framework is entirely robust. The system seamlessly caught all AI failures and produced valid executables via its fallback mechanisms. This work provides a secure, proven foundation for future research. As AI models continue to scale in capability, this pipeline will allow them to be safely unleashed on low level systems code, potentially discovering optimizations that human engineered heuristics have overlooked.

## REFERENCES

- [1] C. Lattner and V. Adve, “LLVM: A compilation framework for lifelong program analysis and transformation,” in *International Symposium on Code Generation and Optimization*, 2004. *CGO 2004.*, IEEE, 2004, pp. 75 to 86.
- [2] N. P. Lopes *et al.*, “Alive2: Bounded translation validation for LLVM,” in *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, 2021, pp. 65 to 79.
- [3] Qwen Team, “Qwen2.5-Coder: A powerful code large language model,” 2024.